

Operational Models para Verificação de Brokers MQTT com ESBMC

Gustavo Lima
email@exemplo.org

Willian Jean
email@exemplo.org

Paulo Gomes
email@exemplo.org

Resumo—Brokers MQTT são componentes críticos em arquiteturas de Internet das Coisas (IoT) e, por serem frequentemente escritos em linguagem C, estão sujeitos a vulnerabilidades de memória e aritméticas. O *Efficient SMT-Based Context-Bounded Model Checker* (ESBMC) é uma ferramenta de verificação formal capaz de detectar tais defeitos, mas sua aplicação a software de rede é limitada pela ausência de *operational models* para APIs de sockets POSIX. Este trabalho propõe o desenvolvimento desses modelos e sua integração ao ESBMC, tendo o Eclipse Mosquitto como estudo de caso, com o objetivo de viabilizar a detecção automatizada de vulnerabilidades em brokers MQTT.

Palavras-chave—verificação formal, model checking, ESBMC, MQTT, Mosquitto, IoT.

I. INTRODUÇÃO

A. Contexto

O protocolo MQTT (*Message Queuing Telemetry Transport*) é o padrão de fato para comunicação em ambientes de Internet das Coisas (IoT), devido ao seu modelo leve de publicação/assinatura [1]. O componente central de uma rede MQTT é o *broker*, que recebe e roteia todas as mensagens entre dispositivos. O Eclipse Mosquitto [2] é uma das implementações de broker mais adotadas, mantida pela Eclipse Foundation e escrita em linguagem C.

Por ser escrito em C, o Mosquitto herda os riscos inerentes à linguagem: gerência manual de memória, possibilidade de *buffer overflows* e comportamentos indefinidos. Essas características tornam software de rede escrito em C particularmente suscetível a vulnerabilidades de segurança.

O ESBMC (*Efficient SMT-Based Context-Bounded Model Checker*) [3] é uma ferramenta de *Bounded Model Checking* que verifica programas C/C++ traduzindo propriedades de segurança para fórmulas em lógica SMT. O ESBMC é capaz de detectar automaticamente classes de defeitos como *buffer overflow*, *null pointer dereference*, estouro aritmético e vazamentos de memória.

B. Problema

A aplicação do ESBMC a software de rede como o Mosquitto é limitada pela ausência de *operational models* para APIs externas usadas nesse domínio. APIs de sockets POSIX (`socket`, `bind`, `recv`, `send`) e de multiplexação de E/S (`select`, `poll`) não possuem modelos abstratos no ESBMC. Sem esses modelos, a ferramenta aborta a verificação por símbolos indefinidos ou tenta seguir a implementação do sistema operacional, tornando a análise inviável.

O histórico do Mosquitto registra CVEs que poderiam ser detectados por verificação formal: *integer overflow* no campo *Remaining Length* (CVE-2017-7651), vazamento de memória no fluxo de conexão (CVE-2017-7654) e *off-by-one* em rotinas de parsing (CVE-2018-12551). Essas vulnerabilidades envolvem padrões clássicos de defeitos em C que o ESBMC já sabe detectar, mas a barreira de modelagem de APIs de rede impede a análise.

O problema central é: como estender o ESBMC com *operational models* para APIs de rede de modo a viabilizar a detecção automatizada de vulnerabilidades em brokers MQTT escritos em C?

C. Motivação e Justificativa

Brokers MQTT são pontos únicos de falha em arquiteturas IoT. O comprometimento de um broker permite interceptar ou injetar mensagens em toda a rede de dispositivos. Detectar vulnerabilidades antes da implantação reduz o risco e o custo de operação desses sistemas.

Do ponto de vista científico, o catálogo de *operational models* do ESBMC para APIs de rede é incompleto. Preencher essa lacuna beneficia não apenas a análise de brokers MQTT, mas toda uma classe de software de rede. Além disso, os modelos são implementados em arquivos C no diretório de biblioteca do ESBMC, o que permite contribuição direta ao projeto *open source* com baixo risco de regressão.

A escolha do Mosquitto como estudo de caso se justifica por ser *open source*, amplamente documentado e possuir um histórico público de CVEs que servem como oráculo de validação.

D. Objetivos

1) *Objetivo Geral*: Desenvolver *operational models* para APIs de rede no ESBMC e avaliar sua eficácia na detecção de vulnerabilidades no Eclipse Mosquitto.

2) *Objetivos Específicos*:

- 1) Identificar as APIs externas cuja ausência de modelagem impede a verificação do Mosquitto pelo ESBMC.
- 2) Implementar *operational models* para essas APIs no ESBMC.
- 3) Construir *harnesses* de verificação para subsistemas críticos do Mosquitto, incluindo o parser MQTT e o tratamento do pacote CONNECT.

- 4) Reproduzir CVEs históricos do Mosquitto com o ESBMC estendido e comparar os resultados com o ESBMC *vanilla*.
- 5) Avaliar o impacto da extensão em termos de tempo de verificação, consumo de memória e taxa de detecção.

REFERÊNCIAS

- [1] OASIS, “MQTT Version 5.0 – OASIS Standard,” <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>, 2019, accessed: 2026-05-11.
- [2] Eclipse Foundation, “Eclipse Mosquitto: An Open Source MQTT Broker,” <https://mosquitto.org/>, accessed: 2026-05-11.
- [3] L. Cordeiro, B. Fischer, and J. Marques-Silva, “SMT-Based Bounded Model Checking for Embedded ANSI-C Software,” *IEEE Transactions on Software Engineering*, vol. 38, no. 4, pp. 957–974, 2012.